Here are extremely detailed, teacher-level notes for Unit 4: **Polymorphism, Pointers & Virtual Functions**. Each topic is treated as an individual full-page style explanation with clear definitions, need, syntax, and 3 example programs (2 beginner, 1 medium level) to ensure clarity for first-time readers.

---

1. Introduction to Polymorphism

Definition:

Polymorphism is the ability of a function, object, or operator to behave differently based on the context. It allows entities to take multiple forms. In C++, polymorphism is of two types:

- Compile-time (Static) Polymorphism
- Run-time (Dynamic) Polymorphism

Need for Polymorphism:

- Supports code reusability and flexibility.
- Promotes modular and scalable code design.
- Enables implementation of dynamic behavior at runtime.

Syntax:

```
class Base {
public:
   virtual void display() { cout << "Base class"; }
};

class Derived : public Base {
public:
   void display() override { cout << "Derived class"; }
};
```

Example Programs:

Beginner 1: Function with same name, different types

```
#include<iostream>
using namespace std;

void show(int x) { cout << "Integer: " << x << endl; }
void show(float y) { cout << "Float: " << y << endl; }

int main() {
    show(10);     // Integer
    show(5.6f);   // Float
}
```

Beginner 2: Function overriding

```
#include<iostream>
using namespace std;

class Animal {
public:
    virtual void sound() { cout << "Some sound\n"; }
};

class Dog : public Animal {
public:
    void sound() override { cout << "Bark\n"; }
};

int main() {
    Animal *a;
    Dog d;
    a = &d;
    a->sound();
}
```

Medium: Shape Area using function overriding

```
#include<iostream>
using namespace std;

class Shape {
public:
    virtual void area() { cout << "Area of shape\n"; }
```

```
};

class Rectangle : public Shape {
public:
    void area() override { cout << "Area = l*b\n"; }
};

class Circle : public Shape {
public:
    void area() override { cout << "Area = π*r*r\n"; }
};

int main() {
    Shape* s;
    Rectangle r;
    Circle c;
    s = &r; s->area();
    s = &c; s->area();
}
```

---

2. Function Overloading

Definition:

Function overloading allows multiple functions to have the same name with different parameters.

Need:

- Improves readability and usability.
- Provides intuitive interface.

Syntax:

```
void show(int);
void show(double);
```

Example Programs:

Beginner 1:

```
#include<iostream>
using namespace std;

void greet() { cout << "Hello!\n"; }
void greet(string name) { cout << "Hello, " << name << "!\n"; }

int main() {
    greet();
    greet("Aman");
}
```

Beginner 2:

```
#include<iostream>
using namespace std;

int add(int a, int b) { return a + b; }
float add(float a, float b) { return a + b; }

int main() {
    cout << add(5, 3) << endl;
    cout << add(2.5f, 4.3f) << endl;
}
```

Medium: Calculator using overloading

```
#include<iostream>
using namespace std;

class Calculator {
public:
    int calc(int x, int y) { return x + y; }
    float calc(float x, float y) { return x * y; }
    double calc(double x, double y) { return x - y; }
};

int main() {
    Calculator c;
    cout << c.calc(4, 3) << endl;
    cout << c.calc(2.5f, 3.5f) << endl;
    cout << c.calc(7.5, 4.5) << endl;
```

```
}
```

---

3. Operator Overloading

Definition:

Operator overloading enables the use of standard operators (+, -, *, etc.) for user-defined types (like objects).

Need:

- Enhances code clarity when using objects.
- Makes class behave like built-in types.

Rules:

- Only existing operators can be overloaded.
- Cannot overload: `::`, `.*`, `.`, `sizeof`, `?:`

Syntax:

ReturnType operator+(ClassName obj);

Example Programs:

Beginner 1: Overloading +

```
#include<iostream>
using namespace std;

class Complex {
    int real;
public:
    Complex(int r) : real(r) {}
    Complex operator+(Complex c) {
        return Complex(real + c.real);
    }
```

```
    void display() { cout << real << endl; }
};

int main() {
    Complex c1(5), c2(10);
    Complex c3 = c1 + c2;
    c3.display();
}
```

Beginner 2: Overload -

```
#include<iostream>
using namespace std;

class Number {
    int x;
public:
    Number(int a) : x(a) {}
    Number operator-(Number n) { return Number(x - n.x); }
    void show() { cout << x << endl; }
};

int main() {
    Number a(8), b(3);
    Number c = a - b;
    c.show();
}
```

Medium: Complex Number Addition

```
#include<iostream>
using namespace std;

class Complex {
    int real, imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}
    Complex operator+(Complex c) {
        return Complex(real + c.real, imag + c.imag);
    }
    void display() { cout << real << " + " << imag << "i\n"; }
};
```

```
int main() {
    Complex a(2, 3), b(4, 5), c;
    c = a + b;
    c.display();
}
```

---

4. Constructor Overloading

Definition:

Constructor overloading is defining multiple constructors with different signatures in the same class.

Need:

- Provides flexibility in object creation.
- Enables initialization in different ways.

Syntax:

```
class A {
public:
    A();
    A(int);
};
```

Example Programs:

Beginner 1:

```
#include<iostream>
using namespace std;

class Box {
public:
    Box() { cout << "Default Constructor\n"; }
    Box(int l) { cout << "Length = " << l << endl; }
};
```

```
int main() {
    Box b1;
    Box b2(10);
}
```

Beginner 2:

```
#include<iostream>
using namespace std;

class Student {
public:
    Student() { cout << "Unknown Student\n"; }
    Student(string name) { cout << "Name: " << name << endl; }
};

int main() {
    Student s1;
    Student s2("Rahul");
}
```

Medium: Bank Account

```
#include<iostream>
using namespace std;

class Account {
    string name;
    int balance;
public:
    Account() : name("Anonymous"), balance(0) {}
    Account(string n, int b) : name(n), balance(b) {}
    void show() { cout << name << " - Rs. " << balance << endl; }
};

int main() {
    Account a1;
    Account a2("Priya", 5000);
    a1.show();
    a2.show();
}
```

5. Pointers

Definition:

Pointer is a variable that stores the address of another variable.

Need:

- Enables dynamic memory allocation.
- Supports polymorphism and object manipulation.

Syntax:

```
int* ptr;
ptr = &var;
```

Pointer to Objects:

```
ClassName *ptr = &object;
```

this Pointer:

Refers to the current object.

Pointer to Derived Class:

Allows base class pointer to refer to derived object.

Static vs Dynamic Binding:

- **Static:** Resolved at compile-time (e.g., function overloading).
- **Dynamic:** Resolved at runtime (e.g., virtual functions).

Example Programs:

Beginner 1: Pointer to variable

```
#include<iostream>
using namespace std;

int main() {
    int x = 10;
    int* p = &x;
    cout << *p << endl;
}
```

Beginner 2: Pointer to object

```
#include<iostream>
using namespace std;

class Car {
public:
    void drive() { cout << "Driving...\n"; }
};

int main() {
    Car c, *ptr = &c;
    ptr->drive();
}
```

Medium: Base pointer, dynamic binding

```
#include<iostream>
using namespace std;

class Animal {
public:
    virtual void speak() { cout << "Animal speaks\n"; }
};

class Dog : public Animal {
public:
    void speak() override { cout << "Dog barks\n"; }
};

int main() {
    Animal *a;
```

```
    Dog d;
    a = &d;
    a->speak();
}
```

Here are **detailed teacher-level notes** for the topics from **Unit 5: Exception Handling, DMA, and Files** in **Object-Oriented Programming (OOP)**. Each subtopic includes definitions, syntax, detailed explanations, vein diagrams (in textual form), real-life examples, and three sample programs (2 beginner-level and 1 medium-level) where applicable.

🧠 Unit 5: Exception Handling, DMA & File Handling in C++

📌 1. Exception Handling in C++

🔹 **Definition:**

Exception Handling is a mechanism to handle runtime errors in a structured way using `try`, `throw`, and `catch` blocks.

🔹 **Need:**

- Prevents program crashes due to unexpected conditions like division by zero, file errors, memory allocation failures, etc.
- Promotes clean error handling and modular code.

🔹 **Vein Diagram (Textual Format):**

Exception Handling

├── try block → Code that might throw exception

├── throw → Throws the exception

└── catch block → Handles the exception

🔹 **Syntax:**

```
try {
    // Risky code
    if (condition)
        throw exceptionType;  // Throwing an exception
}
catch (exceptionType var) {
    // Handling code
}
```

🔹 **Real-Life Analogy:**

A power socket (try block) might trip if overloaded (throw). The fuse (catch block) protects the appliance by catching the error.

🧪 Programs:

Beginner Example 1: Division by Zero

```
#include <iostream>
using namespace std;

int main() {
```

```
    int a = 10, b = 0;


    try {
        if (b == 0)
            throw "Division by zero error!";
        cout << "Result: " << a / b << endl;
    } catch (const char* msg) {
        cout << "Exception: " << msg << endl;
    }
    return 0;
}
```

Beginner Example 2: Negative Age

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter age: ";
    cin >> age;

    try {
        if (age < 0)
            throw age;
```

```
        cout << "Valid age: " << age << endl;
    } catch (int e) {
        cout << "Caught an exception: Invalid age " << e << endl;
    }
}
```

Medium Example: Multiple Catch Blocks

```
#include <iostream>
using namespace std;

int main() {
    try {
        int choice;
        cout << "Enter 1 to throw int, 2 for char: ";
        cin >> choice;

        if (choice == 1) throw 100;
        else if (choice == 2) throw 'A';
        else throw 1.5;
    }
    catch (int x) {
        cout << "Caught integer: " << x << endl;
    }
    catch (char y) {
```

```
        cout << "Caught character: " << y << endl;
    }
    catch (...) {
        cout << "Caught unknown exception!" << endl;
    }
}
```

📌 2. Dynamic Memory Allocation using `new` and `delete`

🔹 **Definition:**

Dynamic Memory Allocation (DMA) refers to allocating memory during runtime using `new` and deallocating it using `delete`.

🔹 **Syntax:**

```
int* ptr = new int;     // Allocating memory
*ptr = 10;              // Using memory
delete ptr;             // Deallocating memory
```

🔹 **Vein Diagram:**

Dynamic Memory Allocation

├── new → Allocates memory

└── delete → Frees memory

🔹 **Real-Life Analogy:**

Like booking a hotel room (`new`) and vacating it (`delete`) when you're done.

🧪 Programs:

Beginner Example 1: Allocate Integer

```
#include <iostream>

using namespace std;


int main() {

    int* ptr = new int;

    *ptr = 25;

    cout << "Value: " << *ptr << endl;

    delete ptr;

    return 0;

}
```

Beginner Example 2: Allocate Array

```
#include <iostream>

using namespace std;


int main() {

    int* arr = new int[5];

    for (int i = 0; i < 5; i++)
```

```
        arr[i] = i + 1;

    for (int i = 0; i < 5; i++)
        cout << arr[i] << " ";

    delete[] arr;
    return 0;
}
```

Medium Example: Sum of N Numbers using DMA

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int* arr = new int[n];
    int sum = 0;

    for (int i = 0; i < n; i++) {
        cout << "Enter value " << i + 1 << ": ";
        cin >> arr[i];
```

```
        sum += arr[i];
    }

    cout << "Sum: " << sum << endl;
    delete[] arr;
    return 0;
}
```

📌 3. File Handling in C++

🔹 **Introduction:**

C++ provides file I/O through the `<fstream>` header which includes:

- `ifstream` (for reading)
- `ofstream` (for writing)
- `fstream` (for both)

🔹 **Hierarchy of File Stream Classes:**

```
ios
├── istream
│   └── ifstream
├── ostream
│   └── ofstream
```

└── iostream

└── fstream

- **File Opening Modes:**

Mode	Description
ios::in	Open for reading
ios::out	Open for writing
ios::app	Append to end
ios::binary	Binary mode
ios::ate	Start at end of file
ios::trunc	Truncate file

- **Syntax:**

```
ofstream fout("file.txt"); // write

ifstream fin("file.txt");  // read

fstream file("file.txt", ios::in | ios::out); // both
```

🔹 **Real-Life Analogy:**

Think of a file as a diary. `ofstream` is like writing into it, `ifstream` is like reading, and `fstream` allows both.

🧪 Programs:

Beginner Example 1: Write to File

```
#include <fstream>

using namespace std;


int main() {

    ofstream file("example.txt");

    file << "Hello File!";

    file.close();

    return 0;

}
```

Beginner Example 2: Read from File

```
#include <fstream>

#include <iostream>

using namespace std;


int main() {

    ifstream file("example.txt");
```

```
    string line;
    while (getline(file, line)) {
        cout << line << endl;
    }
    file.close();
    return 0;
}
```

Medium Example: Student Data Storage

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream out("student.txt");
    string name;
    int roll;

    cout << "Enter name: ";
    cin >> name;
    cout << "Enter roll no: ";
    cin >> roll;

    out << name << " " << roll << endl;
```

```
    out.close();

    ifstream in("student.txt");
    in >> name >> roll;
    cout << "Read from file: " << name << " " << roll << endl;
    in.close();
    return 0;
}
```

📌 4. Random Access in Files

🔹 **Concept:**

- Allows moving the file pointer to any location.
- `seekg()` and `seekp()` for get and put pointers.
- `tellg()` and `tellp()` to get current position.

🔹 **Syntax:**

```
file.seekg(pos);   // Set read pointer
file.seekp(pos);   // Set write pointer
```

🔹 **Program Example:**

```
#include <iostream>

#include <fstream>

using namespace std;


int main() {

    fstream file("random.txt", ios::out | ios::in | ios::trunc);

    file << "ABCDEFGH";

    file.seekp(3);          // Move to position 3

    file << "Z";            // Replace 'D' with 'Z'

    file.seekg(0);


    char ch;

    while (file.get(ch))

        cout << ch;


    file.close();

    return 0;

}
```

✅ Summary:

Concept	Feature

`try-throw-catch`	Handles runtime errors
`new/delete`	Allocate and free memory during execution
File streams	`ifstream`, `ofstream`, `fstream` for file I/O
File modes	`ios::in`, `ios::out`, `ios::app`, etc.
Random access	`seekg()`, `seekp()` to jump to any file part

Would you like a combined PDF or PowerPoint version of these notes with diagrams?